

Nand2Tetris — Lab 01

Chirag Virdi

Contents

- And
- And16
- DMux
- DMux4Way
- DMux8Way
- Mux
- Mux16
- Mux4Way16
- Mux8Way16
- Not
- Not16
- Or
- Or16
- Or8Way
- Xor
- HalfAdder
- FullAdder
- Add16
- ALU

And

```
And.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And.hdl
/**

* And gate:
* if (a and b) out = 1, else out = 0
*/

CHIP And {
    IN a, b;
    OUT out;

    PARTS:
        Nand (a=a, b=b, out=c );
        Not(in= c , out=out );

}
```

And16

```
And16.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/And16.hdl
/**
 * 16-bit And gate:
 * for i = 0, ..., 15:
 *   out[i] = a[i] And b[i]
 */
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    Nand(a=a[0], b=b[0], out=c);
    Not(in=c , out=out[0]);
    Nand(a=a[1], b=b[1], out=w);
    Not(in=w, out=out[1]);
    Nand(a=a[2], b=b[2], out=s);
    Not(in=s , out=out[2]);
    Nand(a=a[3], b=b[3], out=d);
    Not(in=d , out=out[3]);
    Nand(a=a[4], b=b[4], out=f);
    Not(in=f , out=out[4]);
    Nand(a=a[5], b=b[5], out=e);
    Not(in=e , out=out[5]);
    Nand(a=a[6], b=b[6], out=x);
    Not(in=x , out=out[6]);
    Nand(a=a[7], b=b[7], out=h);
    Not(in=h , out=out[7]);
    Nand(a=a[8], b=b[8], out=p);
    Not(in=p , out=out[8]);
    Nand(a=a[9], b=b[9], out=l);
    Not(in=l , out=out[9]);
    Nand(a=a[10], b=b[10], out=k);
    Not(in=k , out=out[10]);
    Nand(a=a[11], b=b[11], out=o);
    Not(in=o , out=out[11]);
    Nand(a=a[12], b=b[12], out=u);
    Not(in=u , out=out[12]);
    Nand(a=a[13], b=b[13], out=v);
    Not(in=v , out=out[13]);
    Nand(a=a[14], b=b[14], out=j);
    Not(in=j , out=out[14]);
    Nand(a=a[15], b=b[15], out=t);
    Not(in=t , out=out[15]);

}
```

DMux

```
DMux.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux.hdl
/**
 * Demultiplexor:
 * [a, b] = [in, 0] if sel = 0
 *
 * [0, in] if sel = 1
 */
CHIP DMux {
    IN in, sel;
    OUT a, b;

    PARTS:
    Not(in=sel , out=w);
    And(a=in, b=w, out=a );
    And(a=in, b=sel , out=b );
}
```

DMux4Way

```
DMux4Way.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux4Way.hdl
/**
 * 4-way demultiplexor:
 * [a, b, c, d] = [in, 0, 0, 0] if sel = 00

*           [0, in, 0, 0] if sel = 01
*           [0, 0, in, 0] if sel = 10
*           [0, 0, 0, in] if sel = 11
 */
CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
    DMux(in=in, sel=sel[1], a=in0, b=in1);
    DMux(in=in0, sel=sel[0], a=a, b=b);
    DMux(in=in1, sel=sel[0], a=c, b=d);

}
```

DMux8Way

DMux8Way.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/DMux8Way.hdl
/**
 * 8-way demultiplexor:
 * [a, b, c, d, e, f, g, h] = [in, 0, 0, 0, 0, 0, 0, 0] if sel = 000
 *
 * [0, in, 0, 0, 0, 0, 0, 0] if sel = 001 [0,
 * 0, in, 0, 0, 0, 0, 0] if sel = 010 [0, 0, 0,
 * in, 0, 0, 0, 0] if sel = 011 [0, 0, 0, 0,
 * in, 0, 0, 0] if sel = 100 [0, 0, 0, 0, 0,
 * in, 0, 0] if sel = 101 [0, 0, 0, 0, 0, 0,
 * in, 0] if sel = 110 [0, 0, 0, 0, 0, 0, 0,
 * in] if sel = 111
 */
CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
    DMux(in=in, sel=sel[2], a=abcd, b=efgh);
    DMux4Way(in=abcd, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=efgh, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

Mux

```
Mux.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux.hdl
/**

 * Multiplexor:
 * if (sel = 0) out = a, else out = b
 */
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
    Not(in=sel,      out=x);
    And(a=x,   b=a,   out=w1);
    And(a=sel, b=b , out=w2 );
    Or(a=w1 , b=w2, out=out );
}
```

Mux16

```
Mux16.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux16.hdl
/**
 * 16-bit multiplexor:
 * for i = 0, ..., 15:

    * if (sel = 0) out[i] = a[i], else out[i] = b[i]
 */
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];

    PARTS:
    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);

}
```


Mux4Way16

```
Mux4Way16.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux4Way16.hdl
/**
 *
 *
 * 4-way 16-bit multiplexor:
 * out = a if sel = 00
 *      b if sel = 01
 */
      c if sel = 10
      d if sel = 11

CHIPMux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=ab);
    Mux16(a=c, b=d, sel=sel[0], out=cd);
    Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

Mux8Way16

```
Mux8Way16.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Mux8Way16.hdl
/**
 * 8-way 16-bit multiplexor:
 * out = a if sel = 000

 *      b if sel = 001
 *      c if sel = 010
 *      d if sel = 011
 *      e if sel = 100
 *      f if sel = 101
 *      g if sel = 110
 *      h if sel = 111
 */
CHIP Mux8Way16 {
    IN a[16], b[16], c[16], d[16],
    e[16], f[16], g[16], h[16],
    sel[3];
    OUT out[16];

    PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=abcd);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=efgh);
    Mux16(a=abcd, b=efgh, sel=sel[2], out=out);

}
```

Not

```
Not.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Not.hdl
/**
 * Not gate:
 * if (in) out = 0, else out = 1
 */
CHIP Not {

    IN in;
    OUT out;

    PARTS:
        Nand(a=in , b=in, out= out);
}
```

Not16

```
Not16.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/01/Not16.hdl
/**
 * 16-bit Not gate:
 * for i = 0, ..., 15:
 *   out[i] = Not(a[i])
 */
CHIP Not16 {

    IN in[16];
    OUT out[16];

    PARTS:
    Not(in=in[0] ,out=out[0] );
    Not(in=in[1],   out=out[1]);
    Not(in=in[2],   out=out[2]);
    Not(in=in[3],   out=out[3]);
    Not(in=in[4],   out=out[4]);
    Not(in=in[5],   out=out[5]);
    Not(in=in[6],   out=out[6]);
    Not(in=in[7],   out=out[7]);
    Not(in=in[8],   out=out[8]);
    Not(in=in[9],   out=out[9]);
    Not(in=in[10],  out=out[10]);
    Not(in=in[11],  out=out[11]);
    Not(in=in[12],  out=out[12]);
    Not(in=in[13],  out=out[13]);
    Not(in=in[14],  out=out[14]);
    Not(in=in[15],  out=out[15]);

}
```

Or

```
Or.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or.hdl
/**
 * Or gate:
 * if (a or b) out = 1, else out = 0
 */
CHIP Or {
    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a , b=a, out=x);
    Nand(a=b, b=b, out=y );
    Nand(a=x, b=y , out=out);

}
```

Or16

Or16.hdl

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or16.hdl
```

```
/**
```

```
 * 16-bit Or gate:
 * for i = 0, ..., 15:
 *   out[i] = a[i] Or b[i]
```

```
 */
```

```
CHIP Or16 {
```

```
    IN a[16], b[16];
    OUT out[16];
```

```
    PARTS:
```

```
    Or(a=a[0] , b=b[0] , out=out[0] );
    Or(a=a[1] , b=b[1] , out=out[1] );
    Or(a=a[2] , b=b[2] , out=out[2] );
    Or(a=a[3] , b=b[3] , out=out[3] );
    Or(a=a[4] , b=b[4] , out=out[4] );
    Or(a=a[5] , b=b[5] , out=out[5] );
    Or(a=a[6] , b=b[6] , out=out[6] );
    Or(a=a[7] , b=b[7] , out=out[7] );
    Or(a=a[8] , b=b[8] , out=out[8] );
    Or(a=a[9] , b=b[9] , out=out[9] );
    Or(a=a[10] , b=b[10] , out=out[10] );
    Or(a=a[11] , b=b[11] , out=out[11] );
    Or(a=a[12] , b=b[12] , out=out[12] );
    Or(a=a[13] , b=b[13] , out=out[13] );
    Or(a=a[14] , b=b[14] , out=out[14] );
    Or(a=a[15] , b=b[15] , out=out[15] );
```

```
}
```

Or8Way

```
Or8Way.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Or8Way.hdl
/**
 * 8-way Or gate:

 * out = in[0] Or in[1] Or ... Or in[7]
 */
CHIP Or8Way {
    IN in[8];
    OUT out;

    PARTS:
    Or(a=in[0], b=in[1], out=t1);
    Or(a=t1,    b=in[2],  out=t2);
    Or(a=t2,    b=in[3],  out=t3);
    Or(a=t3,    b=in[4],  out=t4);
    Or(a=t4,    b=in[5],  out=t5);
    Or(a=t5,    b=in[6],  out=t6);
    Or(a=t6,    b=in[7], out=out);

}
```

Xor

```
Xor.hdl
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/1/Xor.hdl
/**
 * Exclusive-or gate:
 * if ((a and Not(b)) or (Not(a) and b)) out = 1, else out = 0
 */
CHIP Xor {

    IN a, b;
    OUT out;

    PARTS:
    Nand(a=a , b=b, out=w );
    Nand(a=a , b=w , out=v );
    Nand(a=b , b=w , out=x );
    Nand(a=v, b=x , out=out );

}
```


HalfAdder

HalfAdder.hdl

```
CHIP HalfAdder {  
  IN a, b;  
  OUT sum, carry;  
  
  PARTS:  
    Xor(a=a, b=b, out=sum);  
    And(a=a, b=b, out=carry);  
}
```

FullAdder

FullAdder.hdl

```
CHIP FullAdder {  
  
    IN a, b, c;  
    OUT sum, carry;  
  
    PARTS:  
    HalfAdder(a=a, b=b, sum=sum1, carry=carry1);  
    HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);  
    Or(a=carry1, b=carry2, out=carry);  
}
```

Add16

Add16.hdl

```
CHIP Add16 {

    IN a[16], b[16];
    OUT out[16];

    PARTS:

        // Bit 0: Half Adder
        HalfAdder(
            a=a[0],
            b=b[0],
            sum=out[0],
            carry=c1
        );

        // Bits 1-15: Full Adders
        FullAdder(
            a=a[1],
            b=b[1],
            c=c1,
            sum=out[1],
            carry=c2
        );

        FullAdder(
            a=a[2],
            b=b[2],
            c=c2,
            sum=out[2],
            carry=c3
        );

        FullAdder(
            a=a[3],
            b=b[3],
            c=c3,
            sum=out[3],
            carry=c4
        );

        FullAdder(
            a=a[4],
            b=b[4],
            c=c4,
            sum=out[4],
            carry=c5
        );

        FullAdder(
            a=a[5],
            b=b[5],
            c=c5,
            sum=out[5],
```

```
        carry=c6
    );

    FullAdder(
        a=a[6],
        b=b[6],
        c=c6,
        sum=out[6],
        carry=c7
    );

    FullAdder(
        a=a[7],
        b=b[7],
        c=c7,
        sum=out[7],
        carry=c8
    );

    FullAdder(
        a=a[8],
        b=b[8],
        c=c8,
        sum=out[8],
        carry=c9
    );

    FullAdder(
        a=a[9],
        b=b[9],
        c=c9,
        sum=out[9],
        carry=c10
    );

    FullAdder(
        a=a[10],
        b=b[10],
        c=c10,
        sum=out[10],
        carry=c11
    );

    FullAdder(
        a=a[11],
        b=b[11],
        c=c11,
        sum=out[11],
        carry=c12
    );

    FullAdder(
        a=a[12],
        b=b[12],
        c=c12,
        sum=out[12],
```

```
        carry=c13
    );

    FullAdder(
        a=a[13],
        b=b[13],
        c=c13,
        sum=out[13],
        carry=c14
    );

    FullAdder(
        a=a[14],
        b=b[14],
        c=c14,
        sum=out[14],
        carry=c15
    );

    FullAdder(
        a=a[15],
        b=b[15],
        c=c15,
        sum=out[15],
        carry=c16
    );
}
```

ALU

ALU.hdl

```
/**
 * Hack ALU
 */

CHIP ALU {
    IN
        x[16], y[16],
        zx, nx, zy, ny, f, no;

    OUT
        out[16],
        zr,
        ng;

    PARTS:

        // x preprocessing
        Mux16(a=x, b=false, sel=zx, out=xZeroed);
        Not16(in=xZeroed, out=xNot);
        Mux16(a=xZeroed, b=xNot, sel=nx, out=xProcessed);

        // y preprocessing
        Mux16(a=y, b=false, sel=zy, out=yZeroed);
        Not16(in=yZeroed, out=yNot);
        Mux16(a=yZeroed, b=yNot, sel=ny, out=yProcessed);

        // AND or ADD
        And16(a=xProcessed, b=yProcessed, out=andResult);
        Add16(a=xProcessed, b=yProcessed, out=addResult);

        Mux16(
            a=andResult,
            b=addResult,
            sel=f,
            out=preOut
        );

        // Final negation
        Not16(in=preOut, out=outNot);

        Mux16(
            a=preOut,
            b=outNot,
            sel=no,
            out=out
        );

        // zr = 1 when out == 0
        Or8Way(in=out[0..7], out=orLow);
        Or8Way(in=out[8..15], out=orHigh);
        Or(a=orLow, b=orHigh, out=orAll);
        Not(in=orAll, out=zr);
    }
}
```

```
// ng = MSB of output
And(a=out[15], b=true, out=ng);
}
```